

System Design Fundamentals

A Complete Guide

5 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:
<https://snehaliteng.github.io/tutorials/system-design-fundamentals/>

1. Introduction

What is System Design?

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy specified requirements. It is the bridge between a product idea and a working implementation at scale.

A well-designed system handles millions of users, processes petabytes of data, and maintains high availability — all while keeping costs manageable and performance predictable.

Why is System Design Important?

- **Scale:** A design that works for 100 users often breaks at 1 million. System design teaches you to plan for growth.
- **Cost Efficiency:** Choosing the right database, caching strategy, or communication protocol can save thousands in infrastructure.
- **Reliability:** Anticipating failures (server crashes, network partitions, traffic spikes) and building resilience into the system.
- **Interview Success:** System design rounds at FAANG and top tech companies evaluate your ability to think holistically about trade-offs, scale, and architecture.
- **Career Growth:** Senior+ engineers are expected to own design decisions, lead architecture discussions, and mentor teams on system thinking.

Evolution of System Design

Era	Typical Architecture	Scale
2000s	Monolithic app + single SQL database	Thousands of users
2010s	Multi-tier (web/app/db), basic caching	Millions of users
2020s	Microservices, event-driven, CDN, NoSQL, cloud-native	Billions of users

Course Navigation

This guide is divided into four parts:

- **Part I: Foundations** — Networking, protocols, architecture patterns, web concepts. Build the vocabulary needed for design discussions.
- **Part II: Core Architecture & Scaling** — Scalability, storage, performance, reliability, security. The building blocks of any large system.
- **Part III: Interview Blueprint & Case Studies** — A repeatable 4-step approach, followed by 13 real-world case studies.
- **Part IV: Wrapping Up** — Final prep, mindset, and resources to continue learning.

Exercise: Think of an application you use daily (e.g., WhatsApp, YouTube, Uber). List three things that would break if it suddenly had 100x more users tomorrow. For each, write a one-sentence hypothesis about how you might fix it.

2. Networking & Communication

IP (Internet Protocol)

Every device on a network is identified by an IP address (IPv4: 32-bit, e.g., `192.168.1.1`; IPv6: 128-bit). IP handles routing packets from source to destination, but does not guarantee delivery, ordering, or error correction — that is left to higher-layer protocols.

DNS (Domain Name System)

DNS translates human-readable domain names (`example.com`) into IP addresses. It uses a hierarchical system: root servers → TLD servers (.com, .org) → authoritative name servers. DNS caching at browsers, OS, ISPs, and CDNs reduces latency.

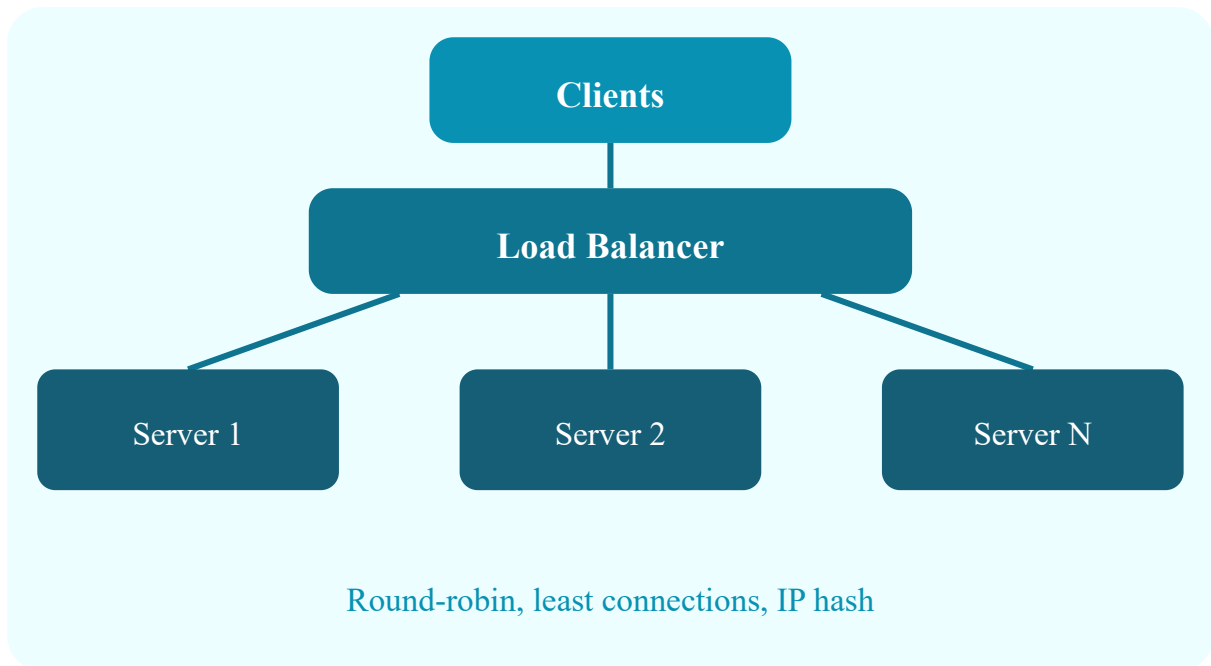
```
# Simplified DNS resolution flow
User types "api.example.com"
Browser checks local cache → OS cache → recursive resolver
Recursive resolver queries root → .com TLD → example.com authoritative
Authoritative server returns IP 93.184.216.34
Browser opens TCP connection to that IP
```

Client-Server Model

The client sends a request; the server processes it and sends a response. This is the foundation of web communication. Variations include thin clients (browser), thick clients (mobile app), and peer-to-peer models.

Proxies

- **Forward Proxy:** Sits between clients and the internet. Used for content filtering, anonymization, and caching. Clients configure their proxy settings explicitly.
- **Reverse Proxy:** Sits in front of servers. Accepts client requests and forwards them to backend servers. Used for load balancing, SSL termination, caching, and hiding internal server topology. Common tools: Nginx, HAProxy, Envoy.



Load Balancing

Load balancers distribute incoming traffic across multiple backend servers to ensure no single server is overwhelmed. Common algorithms:

Algorithm	Behavior	Use Case
Round Robin	Cycles through servers in order	Similar-capacity servers
Least Connections	Sends to server with fewest active connections	Variable request durations
IP Hash	Hashes client IP to pick a server	Session persistence (sticky sessions)
Weighted Round Robin	Servers get traffic proportional to assigned weight	Heterogeneous server capacity

API Gateway

An API gateway sits *between* clients and backend services. It handles cross-cutting concerns: authentication, rate limiting, request routing, protocol translation, logging, and aggregation. Unlike a load balancer (which focuses on traffic distribution), a gateway is often application-aware.

Common tools: AWS API Gateway, Kong, Apigee, Azure API Management.

CDN (Content Delivery Network)

A CDN is a distributed network of edge servers that cache static content (images, CSS, JS, videos) geographically closer to users. This reduces latency, offloads origin servers, and can handle traffic spikes. Key concepts: edge node, origin pull, cache TTL, cache invalidation, POP (Point of Presence).

CDN Request Flow

User in Mumbai requests `image.png`

DNS resolves to nearest CDN edge (e.g., Mumbai PoP)

Edge checks cache:

- Hit: serves cached copy (fast, ~5ms)
- Miss: forwards to origin, caches response, serves user

Exercise: Design a global CDN for a video-streaming platform. How do you decide which content to cache at each edge? How do you handle cache invalidation when a video is re-uploaded?

3. Protocols

TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Guaranteed delivery via ACKs/retransmission	Best-effort, no guarantees
Ordering	Sequenced delivery	No ordering guarantees
Speed	Slower (overhead)	Faster (low overhead)
Use Cases	Web (HTTP), email (SMTP), file transfer (FTP)	Streaming video, VoIP, gaming, DNS

```
# TCP 3-way handshake
Client → SYN           → Server
Client ← SYN+ACK       ← Server
Client → ACK           → Server
# Connection established, data transfer begins
```

HTTP / HTTPS

HTTP is the foundational protocol of the web. HTTPS adds TLS encryption on top of HTTP. Key concepts:

- **Methods:** GET (read), POST (create), PUT (replace), PATCH (partial update), DELETE
- **Status codes:** 2xx (success), 3xx (redirect), 4xx (client error), 5xx (server error)
- **Headers:** Metadata like Content-Type, Authorization, Cache-Control
- **HTTP/1.1 vs HTTP/2:** HTTP/2 introduces multiplexing, header compression, server push

- **HTTP/3:** Built on QUIC (UDP-based), reduces head-of-line blocking, faster connection establishment

```
# HTTP Request
GET /api/users/123 HTTP/1.1
Host: example.com
Authorization: Bearer token123
Accept: application/json

# HTTP Response
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 123, "name": "Alice"}
```

REST (Representational State Transfer)

REST is an architectural style for designing networked applications. Key principles:

- **Stateless:** Each request contains all necessary information; no client context stored on the server.
- **Resource-based:** URLs represent resources (`/users/123`), not actions (`/getUser`).
- **Standard methods:** GET, POST, PUT, PATCH, DELETE map to CRUD operations.
- **Representation:** Resources can be represented as JSON, XML, or other formats.

Real-Time Protocols

- **WebSocket:** Full-duplex persistent connection over TCP. Used for chat, live notifications, real-time dashboards.
- **SSE (Server-Sent Events):** Unidirectional server-to-client streaming over HTTP. Simpler than WebSocket; good for live feeds, notifications.
- **WebRTC:** Peer-to-peer real-time communication (audio, video, data). Used for video calls, conferencing.

Modern API Protocols

- **gRPC:** High-performance RPC framework using Protocol Buffers and HTTP/2. Supports streaming, bidirectional communication. Used heavily in microservices.
- **GraphQL:** Query language that lets clients request exactly the data they need. Reduces over-fetching and under-fetching. Single endpoint, complex queries.

```
# gRPC service definition (Protocol Buffers)
service UserService {
  rpc GetUser (GetUserRequest) returns (User);
  rpc ListUsers (ListUsersRequest) returns (stream User);
}

message GetUserRequest { string user_id = 1; }
```

Exercise: Your team is building a real-time collaborative document editor (like Google Docs). Which protocols would you use for: a) sending edits between users, b) loading the document from the server, c) notifying users of presence (who is editing). Justify each choice.

4. Architecture Patterns

Multi-Tier Architecture

Also called n-tier architecture, this pattern separates an application into logical layers:

- **1-Tier:** All logic + data in a single machine. Simple but not scalable.
- **2-Tier:** Client directly communicates with database. Still limited.
- **3-Tier:** Presentation (UI) → Application (business logic) → Data (database). The most common pattern for web apps.
- **N-Tier:** Additional layers like cache tier, message queue tier, CDN tier, etc.

```
# 3-Tier Request Flow
```

```
Browser (Presentation) → sends HTTP request
```

```
Load Balancer → routes to Application Server
```

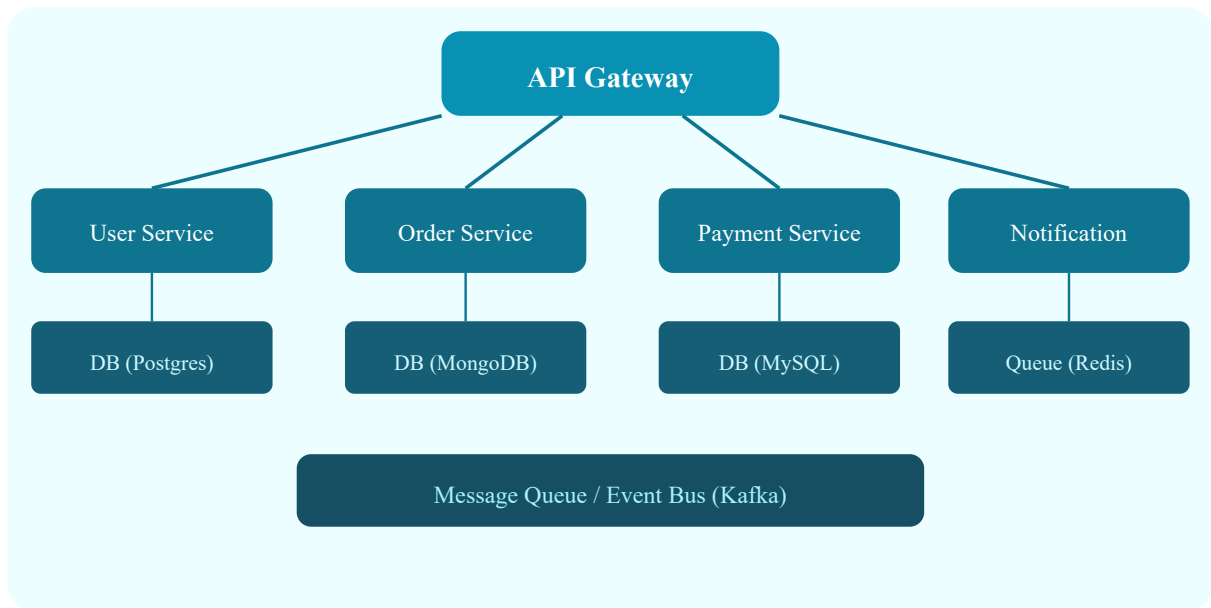
```
Application Server (Business Logic) → processes request
```

```
Application Server queries Database (Data Tier)
```

```
Response flows back through the same chain
```

Microservices Architecture

Microservices decompose an application into small, independently deployable services, each responsible for a specific business capability.



Key characteristics:

- **Decentralized:** Each service owns its data, logic, and deployment.
- **Independent scaling:** Scale only the services under load.
- **Technology diversity:** Different services can use different languages/DBs.
- **Resilience:** Failure in one service does not cascade.
- **Complexity:** Requires service discovery, distributed tracing, API gateways, event buses, and DevOps maturity.

Microservices vs Monolith

Factor	Monolith	Microservices
Deployment	Single deploy unit	Independent deploys
Scaling	Scale entire app	Scale per service
Development	Simple to start	Complex coordination
Testing	End-to-end easier	Contract testing needed
Team autonomy	Low (shared codebase)	High (per-team ownership)

Event-Driven Architecture

Components communicate through events — messages published to an event bus. Producers emit events without knowing who consumes them. Consumers subscribe to relevant events.

```
# Event-Driven Flow
User places order → Order Service publishes "order.created" event
Event Bus (Kafka/RabbitMQ) distributes the event
Inventory Service consumes → reserves stock
Notification Service consumes → sends confirmation email
Analytics Service consumes → records order data
```

Benefits: Loose coupling, asynchronous processing, excellent for workflows spanning multiple services.

Challenges: Event ordering, exactly-once processing, debugging event chains, eventual consistency.

Service Mesh

A service mesh (e.g., Istio, Linkerd) adds a sidecar proxy to each service instance to handle service-to-service communication, observability, and security. It separates these concerns from application code.

CQRS (Command Query Responsibility Segregation)

Separates read and write operations into different models. Writes go through commands; reads go through separate queries — possibly using different data stores or schemas optimized for each purpose.

Exercise: You are designing the backend for an e-commerce platform. Identify which parts would benefit from microservices (e.g., product catalog, checkout, payments) and which might remain monolithic. Draw the service boundaries and describe how they communicate.

5. Web Concepts

Sessions

HTTP is stateless — each request is independent. Sessions allow servers to maintain state across multiple requests from the same client.

Approaches:

- **Cookie-based sessions:** Server stores session data; client holds a session ID cookie. On each request, the server looks up the session by ID.
- **JWT (JSON Web Tokens):** Client holds a signed token containing user info and claims. No server-side session store needed. Tokens can be verified without DB lookup.
- **Token-based (opaque tokens):** Server issues a random token; client sends it via Authorization header. Server looks up the token in a store (Redis, DB).

```
# Session-based authentication
POST /login { "username": "alice", "password": "****" }
Server validates credentials, creates session, returns:
Set-Cookie: session_id=abc123; HttpOnly; Secure

GET /profile
Cookie: session_id=abc123
Server looks up session, finds user_id=42, returns profile
```

Serialization

Serialization converts an in-memory object into a format that can be stored or transmitted. Deserialization reverses this.

Format	Human-Readable	Binary	Schema Required	Performance
JSON	Yes	No	No	Moderate
XML	Yes	No	Optional	Slower
Protocol Buffers	No	Yes	Yes (.proto)	Fast
Avro	No	Yes	Yes (inline/separate)	Fast
MessagePack	No	Yes	No	Fast

```
# JSON Serialization (Python dict → JSON string)
{"user": {"id": 42, "name": "Alice", "roles": ["admin"]}}

# Protocol Buffers (compact binary, ~40% smaller than JSON)
message User { int32 id = 1; string name = 2; repeated string roles = 3; }
```

CORS (Cross-Origin Resource Sharing)

CORS is a browser security mechanism that restricts web pages from making requests to a different domain than the one that served the page. The server specifies allowed origins via HTTP headers.

```
# Server response header to allow cross-origin requests
Access-Control-Allow-Origin: https://myapp.com
Access-Control-Allow-Methods: GET, POST, PUT
Access-Control-Allow-Headers: Content-Type, Authorization

# Preflight request (OPTIONS) for non-simple requests
OPTIONS /api/data
Origin: https://myapp.com
Access-Control-Request-Method: DELETE
```

Idempotency

An operation is idempotent if performing it multiple times has the same effect as performing it once. Crucial for distributed systems where retries are common.

- **GET, PUT, DELETE** are idempotent by nature.
- **POST** is generally *not* idempotent (creates new resources each time).
- **Idempotency key:** Clients send a unique key; servers detect duplicates and skip processing.

Exercise: Your API receives duplicate payment requests due to network retries. Design an idempotency mechanism using an idempotency key. How do you store it? When do you expire it? What happens if the server crashes mid-processing?